



Knowledge Based Systems

Laboratory activity

Name: Marcu Cristian Petric
Group: 30441
Email: marcupetric10@gmail.com

Teaching Assistant: Marius Adrian Stoica
Stoica.Ma.Marius@student.utcluj.ro



Contents

1	Determine Scope	3
2	Consider Reuse	4
3	Enumerate Important Terms	5
4	Define Classes	6
5	Define Roles	8
6	Role Constraints	11
7	Create Instances	12
8	Check for Anomalies	13
9	Ontology evaluation	14
A	Your code	16

Chapter 1

Determine Scope

Purpose: The ontology models a modern streaming platform ecosystem. It integrates production data (creators, release dates) with a robust content catalog (Films, Series, Episodes, Age Ratings) and consumer access data (streaming platforms, subscriptions, viewing history) to support intelligent content recommendation and access querying.

Competency Questions:

1. Which films and series are currently available on a specific streaming platform (e.g., Netflix)?
2. Which subscribers meet the age requirements to view specific “TV-MA” or “R” rated content?
3. How many episodes belonging to a specific series has a user watched?
4. What is the average rating of content within the “Science Fiction” genre?
5. Which subscribers are paying for a Premium subscription plan but haven’t watched any content this month?

Chapter 2

Consider Reuse

For this project, I have chosen to extend an existing ontology rather than building one from scratch.

- **The ontology used:** The Film Ontology.
- **Source/Link:** <http://semantics.id/ns/example/film/>
- **Author(s):** Fajar J. Ekaputra (Contributors: Filip Kovacevic, Laura Waltersdorfer, Marta Sabou).
- **Extension Strategy:** The base ontology effectively models film creation (artists, studios). I will extend it by adding a “Consumer/Streaming” domain to model where these films and series are hosted, the episodic nature of content, robust content classification, and user viewing habits.

Chapter 3

Enumerate Important Terms

The relevant terms for both the base ontology and the new extensions are listed below:

- **Nouns (candidate classes):**

- *From Base:* Film, Actor, Director, Soundtrack, Artwork, Person, Composer, Film-Studio.
- *New Extensions:* Series, Episode, Classification, Genre, AgeRating, Subscriber, StreamingPlatform, SubscriptionPlan.

- **Verbs (candidate object properties):**

- *From Base:* hasActor, hasDirector, hasCrew, prequel, sequel.
- *New Extensions:* availableOn, watched, likes, subscribedTo, hasPlan, belongsToSeries, hasGenre, hasAgeRating.

- **Descriptive nouns (candidate data properties):**

- *From Base:* releaseYear, fullName, dateOfBirth, budget.
- *New Extensions:* subscriberAge, subscriptionCost, imdbRating.

Chapter 4

Define Classes

The class hierarchy based on the base ontology and our streaming platform extensions, totaling exactly 40 classes to meet the project requirements:

- **Artwork**
 - Film
 - * ShortFilm (*New*)
 - Series (*New*)
 - * Miniseries (*New*)
 - Season (*New*)
 - Episode (*New*)
 - Soundtrack
 - Trailer (*New*)
- **Classification** (*New*)
 - Genre
 - AgeRating (*New*)
- **Person** (foaf:Person)
 - Director
 - Composer
 - ScriptWriter
 - Producer (*New*)
 - Cinematographer (*New*)
 - Editor (*New*)
 - CostumeDesigner (*New*)
 - Performer
 - * Actor
 - VoiceActor (*New*)
 - Subscriber (*New*)
- **FilmStudio**

- **StreamingPlatform** (*New*)
- **SubscriptionPlan** (*New*)
 - BasicPlan (*New*)
 - PremiumPlan (*New*)
 - FamilyPlan (*New*)
 - AdSupportedPlan (*New*)
 - StudentPlan (*New*)
- **Country** (*New*)
- **Review** (*New*)
- **Award** (*New*)
 - Oscar (*New*)
 - Emmy (*New*)
 - * BestComedy (*New*)
 - * BestDrama (*New*)
 - GoldenGlobe (*New*)
 - PalmeDOr (*New*)

Chapter 5

Define Roles

Object properties (relations between classes) - Total: 21

- `hasActor` (Film $\rightarrow$ Actor) (*Base*)
- `hasDirector` (Film $\rightarrow$ Director) (*Base*)
- `hasCrew` (Film $\rightarrow$ Person) (*Base*)
- `hasComposer` (Film $\rightarrow$ Composer) (*Base*)
- `hasScriptWriter` (Film $\rightarrow$ ScriptWriter) (*Base*)
- `hasPerformer` (Artwork $\rightarrow$ Performer) (*Base*)
- `hasGenre` (Film $\rightarrow$ Genre) (*Base*)
- `hasFilmStudio` (Film $\rightarrow$ FilmStudio) (*Base*)
- `hasSoundtrack` (Film $\rightarrow$ Soundtrack) (*Base*)
- `prequel` (Film $\rightarrow$ Film) (*Base*)
- `sequel` (Film $\rightarrow$ Film) (*Base*)
- `friendOf` (Person $\rightarrow$ Person) (*Base*)
- `knows` (Person $\rightarrow$ Person) (*Base*)
- `watched` (Subscriber $\rightarrow$ Artwork) (*New*)
- `likes` (Subscriber $\rightarrow$ Artwork) (*New*)
- `availableOn` (Artwork $\rightarrow$ StreamingPlatform) (*New*)
- `belongsToSeries` (Episode $\rightarrow$ Series) (*New*)
- `hasPlan` (Subscriber $\rightarrow$ SubscriptionPlan) (*New*)
- `subscribedTo` (Subscriber $\rightarrow$ StreamingPlatform) (*New*)
- `hasAgeRating` (Artwork $\rightarrow$ AgeRating) (*New*)
- `hasAward` (Artwork $\rightarrow$ Award) (*New*)

Data properties (attributes) - Total: 20

- `budget` (Film $\rightarrow$ float) (*Base*)
- `dateOfBirth` (Person $\rightarrow$ date) (*Base*)
- `duration` (Artwork $\rightarrow$ float) (*Base*)
- `establishedDate` (FilmStudio $\rightarrow$ date) (*Base*)
- `fullName` (Person $\rightarrow$ string) (*Base*)
- `gender` (Person $\rightarrow$ string) (*Base*)
- `releaseYear` (Film $\rightarrow$ integer) (*Base*)
- `subscriberAge` (Subscriber $\rightarrow$ integer) (*New*)
- `imdbRating` (Artwork $\rightarrow$ decimal) (*New*)
- `subscriptionCost` (SubscriptionPlan $\rightarrow$ decimal) (*New*)
- `episodeNumber` (Episode $\rightarrow$ integer) (*New*)
- `seasonNumber` (Season $\rightarrow$ integer) (*New*)
- `synopsis` (Artwork $\rightarrow$ string) (*New*)
- `originalLanguage` (Artwork $\rightarrow$ string) (*New*)
- `maxSimultaneousStreams` (SubscriptionPlan $\rightarrow$ integer) (*New*)
- `platformURL` (StreamingPlatform $\rightarrow$ string) (*New*)
- `hasAds` (SubscriptionPlan $\rightarrow$ boolean) (*New*)
- `registrationDate` (Subscriber $\rightarrow$ date) (*New*)
- `awardYear` (Award $\rightarrow$ integer) (*New*)
- `reviewScore` (Review $\rightarrow$ integer) (*New*)

Annotation properties:

- `rdfs:label` (Used for human-readable naming)
- `dc:creator` (Used to track who created the ontology)
- `rdfs:comment` (Used for adding descriptions to new classes)

Stage 5 Extension: Advanced Role Characteristics

To fulfill the logical constraints of the ontology and map the complex relationships within a streaming platform ecosystem, we defined specific use-cases for the advanced OWL relation characteristics:

- **Functional Property:** `hasAgeRating` (`Artwork` $\rightarrow$ `AgeRating`). A specific piece of artwork can only hold one official age classification.
- **Inverse Functional Property:** `wroteReview` (`Subscriber` $\rightarrow$ `Review`). A specific review can uniquely identify its author; a single review cannot be authored by two distinct subscribers.
- **Transitive Property:** `sequel` (`Film` $\rightarrow$ `Film`). If Film C is a sequel to Film B, and Film B is a sequel to Film A, the reasoner infers that Film C is in the sequel lineage of Film A.
- **Symmetric Property:** `coStarredWith` (`Actor` $\rightarrow$ `Actor`). If Actor A co-starred with Actor B, then Actor B automatically co-starred with Actor A.
- **Asymmetric Property:** `nextEpisode` (`Episode` $\rightarrow$ `Episode`). If Episode 2 follows Episode 1, it is mathematically impossible for Episode 1 to follow Episode 2.
- **Reflexive Property:** `inSameUniverse` (`Artwork` $\rightarrow$ `Artwork`). By default, every piece of artwork exists within the same cinematic universe as itself.
- **Irreflexive Property:** `recommendsTo` (`Subscriber` $\rightarrow$ `Subscriber`). A subscriber can recommend content to other users on the platform, but cannot logically send a recommendation to themselves.

Chapter 6

Role Constraints

Chapter 7

Create Instances

Chapter 8

Check for Anomalies

Chapter 9

Ontology evaluation

Bibliography

Appendix A

Your code

Intelligent Systems Group

